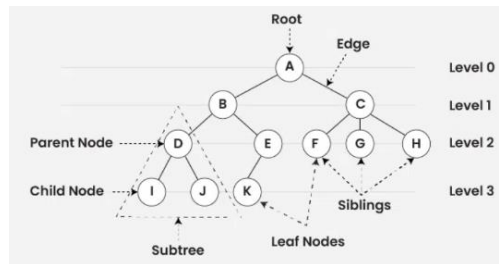


1. Constructing Tree Data Type

We are constructing our own data type, and we will call it a Tree data type. It is of course represented as 'Tree Data Structure' in general, but in the context of scheme, it is an abstract data type that we will define (abstract data types are created from the pov of the user of the data and their implementation details are hidden). Remember, scheme does not know anything about this Tree data type.



A *node* is a point in the tree. Each *node* includes a *datum* (the value shown at the node such as B or C) and also the entire structure under that *datum* and connected to it. So the *B node* contains *D*, *E* and everything under it. Therefore, each *node* is itself a *tree*, so the term *tree* and *node* mean the same thing to us! Generally, we use *Tree* to denote the entire structure that we are manipulating and use *node* for just one piece of the overall structure. Therefore, we can also call the *node* a *subtree*.

The *Root node* (or just the *root*) of a *Tree* is the *node* at the top. Every *Tree* has one *root node*. *Leaf nodes* are the end nodes which have no *children*.

To define this data structure, we can think of each *Tree* as a list of lists. Each list represents a *sub-tree* and the *leaf nodes* are treated as separate lists. So if I want to define *node C*, I can define it as:

(C (F) (G) (H)) , where C is the *datum* and hence the *car* of the list.

Consequently, *Tree A* can be defined as:

(A (B (D (I) (J)) (E (K))) (C (F) (G) (H)))

Since I want the Tree data structure to be an abstract data type and use principles of data abstraction, I will define some constructors and selectors using primitive scheme functions:

(define make-Tree cons) ; constructor (to build the data type)

(define datum car) ; selector (to retrieve information from the data type)

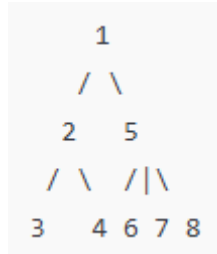
(define children cdr) ; selector (to retrieve information from the data type)

We assume that every *Tree* will have at least one *node* and every *node* has a *datum* and *nodes* can have *children*.

Notice that the selector *children* should return a list of Trees (all the subtrees of a root node). We will call this list of Trees a *forest*. Since *forest* is not a *Tree*, but is in fact a list of *Trees* (or just a sequence

with *Trees* as elements), it is a different data type and we will use standard procedure for manipulating sequences (cons, car, cdr, etc.) ; as a *Tree* is not a sequence, it should be manipulated only with the tree constructors and selectors (make-Tree, datum, children).

To construct the following *Tree*, I can think of a single *sub-tree* and start by constructing that.



So a single tree can be the sub-tree (2 (3) (4)) which can be constructed by:

```
( make-Tree 2 ( list '(3) '(4) ) )
```

To generalize this, I can create a helper procedure that takes elements as arguments and constructs a list where each of the elements is also a list. This helps me define the *leaves* directly.

```
(define (leaves . seq)
```

```
  (map ( lambda (x) (make-Tree x '()) ) seq) )
```

So this takes a variable number of arguments and returns each element as a list so for example, *(leaves 3 4 5) returns ((3) (4) (5))*.

Now I can make my sub-tree as follows:

```
(make-Tree 2 (leaves 3 4))
```

So my overall tree can be constructed as follows:

```
(define Tree
```

```
  (make-Tree 1 (list (make-Tree 2 (leaves 3 4))
```

```
                    (make-Tree 5 (leaves 6 7 8))))))
```

I can also write a simple procedure to check if the *nodes* are *leaf nodes*:

```
(define (is-leaf? Node)
```

```
  (null (children node)) )
```